End-Semester Examination Guide: Tkinter GUI Programming (100 Marks)

Tkinter is Python's standard, built-in library for creating Graphical User Interfaces (GUIs). Unlike standard procedural scripts that execute top-to-bottom and exit, Tkinter operates on an event-driven programming paradigm. In event-driven execution, an infinite loop (`mainloop()`) displays the window and listens continuously for user events (such as button clicks, keyboard input, or mouse movements) to invoke associated handler functions (callbacks).

Question 1 (20 Marks): Fundamental Window Setup, Geometry Managers (`grid, pack, place`), and Display Widgets (`Label, Button`)

(a) [5 Marks] Core Functions and Window Initialization

- **`Tk()`** : Instantiates the root container window.
- **`window.title(string)`** : Sets the text displayed in the window's title bar.
- **`window.geometry("WxH")`** : Configures the initial pixel width and height of the window (e.g., `"400x300"`).
- **`window.config(bg="color") / .configure()`** : Modifies container properties like background color.
- **`window.mainloop()`** : Runs the infinite event loop, keeping the window active on screen and listening for user actions.

(b) [5 Marks] Comparison of Layout Geometry Managers

Tkinter provides three layout managers to place widgets inside a window:

1. **`pack()`** : Stacks widgets linearly (top-to-bottom or left-to-right). Simple, but lacks multi-column positioning.

2. **`grid()`** : Arranges widgets into a two-dimensional grid of rows and columns (starting at `row=0` , `column=0`).

    - `row & column` : Specifies grid cell coordinates.

    - `columnspan & rowspan` : Allows a widget to stretch across multiple columns or rows.

    - `sticky` : Aligns the widget within its cell using compass directions (`N` , `S` , `E` , `W` or `"nw"`).

    - `padx & pady` : Adds external pixel padding around the widget.

3. **`place()`** : Places widgets using absolute coordinate positioning (`x=100` , `y=150`). Precise but difficult to maintain across resizing.

© [5 Marks] Simple Baseline Solution: Basic Window with Label and Button

Python

```
import tkinter as tk

# Handler function (callback)
def handle_click():
    status_label.config(text="Button was clicked!")  # Update label text

# 1. Root Window Setup
root = tk.Tk()
root.title("Basic Tkinter Demo")
root.geometry("350x200")

# 2. Creating Display Widgets
title_label = tk.Label(root, text="Welcome to Tkinter", font=("Arial", 14, "bo
title_label.grid(row=0, column=0, columnspan=2, pady=10)

status_label = tk.Label(root, text="Status: Waiting...", font=("Arial", 10))
status_label.grid(row=1, column=0, columnspan=2, pady=5)
```

```
# 3. Action Button
action_btn = tk.Button(root, text="Click Me", command=handle_click)
action_btn.grid(row=2, column=0, columnspan=2, pady=10)

# 4. Start Event Loop
root.mainloop()
```

(d) [5 Marks] Complex Improvement: Object-Oriented Architecture with State Tracking

Python

```
import tkinter as tk

class InteractiveCounterApp(tk.Tk):
    def __init__(self):
        super().__init__()
        # Window configuration
        self.title("OOP Tkinter Counter")
        self.geometry("400x250")
        self.config(bg="#f0f0f0")

        # State attribute
        self.count = 0

        # Initialize UI components
        self.create_widgets()

    def create_widgets(self):
        # Configure grid column weights for automatic centering
        self.grid_columnconfigure(0, weight=1)
        self.grid_columnconfigure(1, weight=1)

        # Header Label
        self.header_lbl = tk.Label(
            self,
            text="Interactive Counter",
            font=("Helvetica", 16, "bold"),
            bg="#f0f0f0",
```

```
            fg="#333333"
        )
        self.header_lbl.grid(row=0, column=0, columnspan=2, pady=(15, 10), sti

        # Display Counter Label
        self.counter_lbl = tk.Label(
            self,
            text="Current Count: 0",
            font=("Helvetica", 12),
            bg="#ffffff",
            relief="sunken",
            bd=2,
            padx=10,
            pady=5
        )
        self.counter_lbl.grid(row=1, column=0, columnspan=2, pady=10)

        # Increment Button
        self.inc_btn = tk.Button(
            self,
            text="Increment (+1)",
            font=("Helvetica", 10, "bold"),
            bg="#4CAF50",
            fg="white",
            command=self.increment
        )
        self.inc_btn.grid(row=2, column=0, padx=10, pady=15, sticky="e")

        # Reset Button
        self.reset_btn = tk.Button(
            self,
            text="Reset (0)",
            font=("Helvetica", 10, "bold"),
            bg="#f44336",
            fg="white",
            command=self.reset
        )
        self.reset_btn.grid(row=2, column=1, padx=10, pady=15, sticky="w")

    def increment(self):
        self.count += 1
        self.update_display()
```

```
    def reset(self):
        self.count = 0
        self.update_display()

    def update_display(self):
        self.counter_lbl.config(text=f"Current Count: {self.count}")

if __name__ == "__main__":
    app = InteractiveCounterApp()
    app.mainloop()
```

Question 2 (20 Marks): User Input Widgets (`Entry`, `Text`), Control Variables (`StringVar`), and Input Validation

(a) [5 Marks] Entry and Text Widgets

- **`Entry(master)`** : Single-line text input field.
    - `.get()` : Extracts the typed string.
    - `.delete(0, tk.END)` : Clears contents from index 0 to the end.
    - `.insert(0, string)` : Inserts default text at index 0.
- **`Text(master, width, height)`** : Multi-line text field.
    - `.get("1.0", tk.END)` : Retrieves multi-line string starting from line 1, character 0 (`1.0`) to `END` .
    - `.delete("1.0", tk.END)` : Clears all text.

(b) [5 Marks] Tkinter Control Variables

Control variables (`StringVar()` , `IntVar()` , `DoubleVar()` , `BooleanVar()`) store values that automatically bind to GUI widgets, triggering UI updates when changed via `.set(value)` and retrieved via `.get()` .

© [5 Marks] Simple Baseline Solution: Temperature Converter GUI

Python

```
import tkinter as tk

def convert_temp():
    fahrenheit_str = entry_f.get()
    fahrenheit = float(fahrenheit_str)
    celsius = (fahrenheit - 32) * 5 / 9
    label_result.config(text=f"Celsius: {celsius:.2f}°C")

root = tk.Tk()
root.title("Temperature Converter")
root.geometry("300x180")

label_prompt = tk.Label(root, text="Enter Fahrenheit:")
label_prompt.grid(row=0, column=0, padx=10, pady=10)

entry_f = tk.Entry(root)
entry_f.grid(row=0, column=1, padx=10, pady=10)

btn_convert = tk.Button(root, text="Convert", command=convert_temp)
btn_convert.grid(row=1, column=0, columnspan=2, pady=10)

label_result = tk.Label(root, text="Celsius: --")
label_result.grid(row=2, column=0, columnspan=2, pady=10)

root.mainloop()
```

(d) [5 Marks] Complex Improvement: Input Validation with Exception Handling

Python

```
import tkinter as tk
from tkinter import messagebox

class ValidatedConverterApp(tk.Tk):
```

```
    def __init__(self):
        super().__init__()
        self.title("Validated Temperature Converter")
        self.geometry("380x220")
        self.config(bg="#e8ecef")

        self.result_var = tk.StringVar(value="Celsius: --")
        self.build_ui()

    def build_ui(self):
        input_frame = tk.Frame(self, bg="#e8ecef")
        input_frame.pack(pady=15)

        tk.Label(input_frame, text="Fahrenheit (°F):", font=("Arial", 11), bg=

        self.entry_fahrenheit = tk.Entry(input_frame, font=("Arial", 11), widt
        self.entry_fahrenheit.grid(row=0, column=1, padx=5)
        self.entry_fahrenheit.focus()

        self.lbl_result = tk.Label(
            self,
            textvariable=self.result_var,
            font=("Arial", 12, "bold"),
            bg="#ffffff",
            fg="#2c3e50",
            relief="solid",
            bd=1,
            width=25,
            pady=5
        )
        self.lbl_result.pack(pady=10)

        btn_frame = tk.Frame(self, bg="#e8ecef")
        btn_frame.pack(pady=10)

        tk.Button(btn_frame, text="Convert", command=self.process_conversion,
        tk.Button(btn_frame, text="Clear", command=self.clear_fields, bg="#7f8

    def process_conversion(self):
        raw_input = self.entry_fahrenheit.get().strip()
        try:
            if not raw_input:
                raise ValueError("Input field is empty.")
```

```
            fahrenheit = float(raw_input)
            if fahrenheit < -459.67:
                raise ValueError("Temperature below absolute zero (-459.67°F).
            celsius = (fahrenheit - 32) * 5 / 9
            self.result_var.set(f"Celsius: {celsius:.2f}°C")
        except ValueError as ex:
            messagebox.showerror("Invalid Input", f"Error: {ex}")
            self.result_var.set("Celsius: Error")

    def clear_fields(self):
        self.entry_fahrenheit.delete(0, tk.END)
        self.result_var.set("Celsius: --")
        self.entry_fahrenheit.focus()

if __name__ == "__main__":
    app = ValidatedConverterApp()
    app.mainloop()
```

Question 3 (20 Marks): Selection Controls (`Checkbutton`, `Radiobutton`, `Listbox`) and Dialog Popups (`messagebox`)

(a) [5 Marks] Selection Widgets Mechanics

- **`Radiobutton(master, text, variable, value)`** : Allows selection of a single option from a mutually exclusive group bound to a shared variable.
- **`Checkbutton(master, text, variable, onvalue, offvalue)`** : Allows toggle selection (on/off) bound to an `IntVar` or `BooleanVar`.
- **`Listbox(master)`** : Displays a scrollable list of selectable items.
    - `.insert(index, item)` : Adds an item at a specific index.
    - `.curselection()` : Returns a tuple of selected item indices.

(b) [5 Marks] Message Box Popups (`messagebox`)

Imported via `from tkinter import messagebox`:

- `showinfo(title, message)`: Informational dialog with an OK button.
- `showwarning(title, message)`: Warning alert dialog.
- `showerror(title, message)`: Error alert dialog.
- `askyesno(title, message)`: Confirmation prompt returning `True` or `False`.

Question 4 (20 Marks): Event Binding (`.bind()`), Keyboard/Mouse Events, and Canvas Graphics

(a) [5 Marks] Event Binding Mechanics

The `.bind(sequence, func)` method associates an event string with a callback handler function. When the event triggers, Tkinter automatically passes an `Event` object containing event attributes (`event.x`, `event.y`, `event.keysym`) to the handler function.

- Common Event Strings:
  - `<Return>`: User presses the Enter/Return key.
  - `<Button-1>`: Left mouse click.
  - `<B1-Motion>`: Mouse drag with left button held down.

(b) [5 Marks] Canvas Widget Drawing Methods

`Canvas(master, width, height)` creates a region for drawing graphical items:

- `.create_line(...)`: Draws a line segment.
- `.create_rectangle(...)`: Draws a rectangle.
- `.create_oval(...)`: Draws an oval or circle.

- `.delete(item_or_tag)` : Removes drawn items from the canvas.

Would you like me to save or adjust any specific part of this code for your practice session? To resolve the file naming or download issue, the file is being saved or rendered with a default title like **`tkinter.docx`** based on your notebook source content.

If you want to download or export the exact text formatted cleanly as an actual document, you can copy the content below or request it in your preferred layout. Here is the complete, properly structured text content of your Tkinter guide ready for your use:

End-Semester Examination Guide: Tkinter GUI Programming (100 Marks)

Tkinter is Python's standard, built-in library for creating Graphical User Interfaces (GUIs). Unlike standard procedural scripts that execute top-to-bottom and exit, Tkinter operates on an event-driven programming paradigm. In event-driven execution, an infinite loop (`mainloop()`) displays the window and listens continuously for user events (such as button clicks, keyboard input, or mouse movements) to invoke associated handler functions (callbacks).

Question 1 (20 Marks): Fundamental Window Setup, Geometry Managers (`grid` , `pack` , `place`), and Display Widgets (`Label` , `Button`)

(a) [5 Marks] Core Functions and Window Initialization

- **`Tk()`** : Instantiates the root container window.
- **`window.title(string)`** : Sets the text displayed in the window's title bar.
- **`window.geometry("WxH")`** : Configures the initial pixel width and height of the window (e.g., `"400x300"`).

- **`window.config(bg="color") / .configure()`** : Modifies container properties like background color.

- **`window.mainloop()`** : Runs the infinite event loop, keeping the window active on screen and listening for user actions.

(b) [5 Marks] Comparison of Layout Geometry Managers

Tkinter provides three layout managers to place widgets inside a window:

1. **`pack()`** : Stacks widgets linearly (top-to-bottom or left-to-right). Simple, but lacks multi-column positioning.

2. **`grid()`** : Arranges widgets into a two-dimensional grid of rows and columns (starting at `row=0`, `column=0`).

    - `row & column` : Specifies grid cell coordinates.

    - `columnspan & rowspan` : Allows a widget to stretch across multiple columns or rows.

    - `sticky` : Aligns the widget within its cell using compass directions (`N`, `S`, `E`, `W` or `"nw"`).

    - `padx & pady` : Adds external pixel padding around the widget.

3. **`place()`** : Places widgets using absolute coordinate positioning (`x=100`, `y=150`). Precise but difficult to maintain across resizing.

© [5 Marks] Simple Baseline Solution: Basic Window with Label and Button

Python

```
import tkinter as tk

# Handler function (callback)
def handle_click():
    status_label.config(text="Button was clicked!")  # Update label text
```

```
# 1. Root Window Setup
root = tk.Tk()
root.title("Basic Tkinter Demo")
root.geometry("350x200")

# 2. Creating Display Widgets
title_label = tk.Label(root, text="Welcome to Tkinter", font=("Arial", 14, "bo
title_label.grid(row=0, column=0, columnspan=2, pady=10)

status_label = tk.Label(root, text="Status: Waiting...", font=("Arial", 10))
status_label.grid(row=1, column=0, columnspan=2, pady=5)

# 3. Action Button
action_btn = tk.Button(root, text="Click Me", command=handle_click)
action_btn.grid(row=2, column=0, columnspan=2, pady=10)

# 4. Start Event Loop
root.mainloop()
```

(d) [5 Marks] Complex Improvement: Object-Oriented Architecture with State Tracking

Python

```
import tkinter as tk

class InteractiveCounterApp(tk.Tk):
    def __init__(self):
        super().__init__()
        # Window configuration
        self.title("OOP Tkinter Counter")
        self.geometry("400x250")
        self.config(bg="#f0f0f0")

        # State attribute
        self.count = 0

        # Initialize UI components
        self.create_widgets()
```

```
    def create_widgets(self):
        # Configure grid column weights for automatic centering
        self.grid_columnconfigure(0, weight=1)
        self.grid_columnconfigure(1, weight=1)

        # Header Label
        self.header_lbl = tk.Label(
            self,
            text="Interactive Counter",
            font=("Helvetica", 16, "bold"),
            bg="#f0f0f0",
            fg="#333333"
        )
        self.header_lbl.grid(row=0, column=0, columnspan=2, pady=(15, 10), sti

        # Display Counter Label
        self.counter_lbl = tk.Label(
            self,
            text="Current Count: 0",
            font=("Helvetica", 12),
            bg="#ffffff",
            relief="sunken",
            bd=2,
            padx=10,
            pady=5
        )
        self.counter_lbl.grid(row=1, column=0, columnspan=2, pady=10)

        # Increment Button
        self.inc_btn = tk.Button(
            self,
            text="Increment (+1)",
            font=("Helvetica", 10, "bold"),
            bg="#4CAF50",
            fg="white",
            command=self.increment
        )
        self.inc_btn.grid(row=2, column=0, padx=10, pady=15, sticky="e")

        # Reset Button
        self.reset_btn = tk.Button(
            self,
```

```
            text="Reset (0)",
            font=("Helvetica", 10, "bold"),
            bg="#f44336",
            fg="white",
            command=self.reset
        )
        self.reset_btn.grid(row=2, column=1, padx=10, pady=15, sticky="w")

    def increment(self):
        self.count += 1
        self.update_display()

    def reset(self):
        self.count = 0
        self.update_display()

    def update_display(self):
        self.counter_lbl.config(text=f"Current Count: {self.count}")

if __name__ == "__main__":
    app = InteractiveCounterApp()
    app.mainloop()
```

Question 2 (20 Marks): User Input Widgets (`Entry`, `Text`), Control Variables (`StringVar`), and Input Validation

(a) [5 Marks] Entry and Text Widgets

- **`Entry(master)`** : Single-line text input field.
    - `.get()` : Extracts the typed string.
    - `.delete(0, tk.END)` : Clears contents from index 0 to the end.
    - `.insert(0, string)` : Inserts default text at index 0.
- **`Text(master, width, height)`** : Multi-line text field.

- `.get("1.0", tk.END)` : Retrieves multi-line string starting from line 1, character 0 (`1.0`) to `END` .
- `.delete("1.0", tk.END)` : Clears all text.

(b) [5 Marks] Tkinter Control Variables

Control variables (`StringVar()` , `IntVar()` , `DoubleVar()` , `BooleanVar()`) store values that automatically bind to GUI widgets, triggering UI updates when changed via `.set(value)` and retrieved via `.get()` .

© [5 Marks] Simple Baseline Solution: Temperature Converter GUI

Python

```
import tkinter as tk

def convert_temp():
    fahrenheit_str = entry_f.get()
    fahrenheit = float(fahrenheit_str)
    celsius = (fahrenheit - 32) * 5 / 9
    label_result.config(text=f"Celsius: {celsius:.2f}°C")

root = tk.Tk()
root.title("Temperature Converter")
root.geometry("300x180")

label_prompt = tk.Label(root, text="Enter Fahrenheit:")
label_prompt.grid(row=0, column=0, padx=10, pady=10)

entry_f = tk.Entry(root)
entry_f.grid(row=0, column=1, padx=10, pady=10)

btn_convert = tk.Button(root, text="Convert", command=convert_temp)
btn_convert.grid(row=1, column=0, columnspan=2, pady=10)

label_result = tk.Label(root, text="Celsius: --")
label_result.grid(row=2, column=0, columnspan=2, pady=10)
```

```
root.mainloop()
```

(d) [5 Marks] Complex Improvement: Input Validation with Exception Handling

Python

```
import tkinter as tk
from tkinter import messagebox

class ValidatedConverterApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Validated Temperature Converter")
        self.geometry("380x220")
        self.config(bg="#e8ecef")

        self.result_var = tk.StringVar(value="Celsius: --")
        self.build_ui()

    def build_ui(self):
        input_frame = tk.Frame(self, bg="#e8ecef")
        input_frame.pack(pady=15)

        tk.Label(input_frame, text="Fahrenheit (°F):", font=("Arial", 11), bg=

        self.entry_fahrenheit = tk.Entry(input_frame, font=("Arial", 11), widt
        self.entry_fahrenheit.grid(row=0, column=1, padx=5)
        self.entry_fahrenheit.focus()

        self.lbl_result = tk.Label(
            self,
            textvariable=self.result_var,
            font=("Arial", 12, "bold"),
            bg="#ffffff",
            fg="#2c3e50",
            relief="solid",
            bd=1,
            width=25,
            pady=5
```

```
        )
        self.lbl_result.pack(pady=10)

        btn_frame = tk.Frame(self, bg="#e8ecef")
        btn_frame.pack(pady=10)

        tk.Button(btn_frame, text="Convert", command=self.process_conversion,
        tk.Button(btn_frame, text="Clear", command=self.clear_fields, bg="#7f8

    def process_conversion(self):
        raw_input = self.entry_fahrenheit.get().strip()
        try:
            if not raw_input:
                raise ValueError("Input field is empty.")
            fahrenheit = float(raw_input)
            if fahrenheit < -459.67:
                raise ValueError("Temperature below absolute zero (-459.67°F).
            celsius = (fahrenheit - 32) * 5 / 9
            self.result_var.set(f"Celsius: {celsius:.2f}°C")
        except ValueError as ex:
            messagebox.showerror("Invalid Input", f"Error: {ex}")
            self.result_var.set("Celsius: Error")

    def clear_fields(self):
        self.entry_fahrenheit.delete(0, tk.END)
        self.result_var.set("Celsius: --")
        self.entry_fahrenheit.focus()

if __name__ == "__main__":
    app = ValidatedConverterApp()
    app.mainloop()
```

Question 3 (20 Marks): Selection Controls (`Checkbutton`, `Radiobutton`, `Listbox`) and Dialog Popups (`messagebox`)

(a) [5 Marks] Selection Widgets Mechanics

- **`Radiobutton(master, text, variable, value)`** : Allows selection of a single option from a mutually exclusive group bound to a shared variable.
- **`Checkbutton(master, text, variable, onvalue, offvalue)`** : Allows toggle selection (on/off) bound to an `IntVar` or `BooleanVar`.
- **`Listbox(master)`** : Displays a scrollable list of selectable items.
  - `.insert(index, item)` : Adds an item at a specific index.
  - `.curselection()` : Returns a tuple of selected item indices.

(b) [5 Marks] Message Box Popups (`messagebox`)

Imported via `from tkinter import messagebox` :

- `showinfo(title, message)` : Informational dialog with an OK button.
- `showwarning(title, message)` : Warning alert dialog.
- `showerror(title, message)` : Error alert dialog.
- `askyesno(title, message)` : Confirmation prompt returning `True` or `False`.

Question 4 (20 Marks): Event Binding (`.bind()`), Keyboard/Mouse Events, and Canvas Graphics

(a) [5 Marks] Event Binding Mechanics

The `.bind(sequence, func)` method associates an event string with a callback handler function. When the event triggers, Tkinter automatically passes an `Event` object containing event attributes (`event.x` , `event.y` , `event.keysym`) to the handler function.

- Common Event Strings:
  - `<Return>` : User presses the Enter/Return key.

- `<Button-1>` : Left mouse click.
- `<B1-Motion>` : Mouse drag with left button held down.

(b) [5 Marks] Canvas Widget Drawing Methods

`Canvas(master, width, height)` creates a region for drawing graphical items:

- `.create_line(...)` : Draws a line segment.
- `.create_rectangle(...)` : Draws a rectangle.
- `.create_oval(...)` : Draws an oval or circle.
- `.delete(item_or_tag)` : Removes drawn items from the canvas.

Would you like me to save or adjust any specific part of this code for your practice session?